

27/03/25

System.out.printf()

In java, we use System.out.printf() to format our print output in a structured way. The printf method allows us to include placeholders (format specifiers) in a string and replace them with values. It provides more control over the formatting compared to System.out.print() or System.out.println().

Why use System.out.printf()?

1. **Formatted Output:-** It allows to control how data should display.
2. **String Interpolation:-** Support placeholders like %d, %f, %s.
3. **Alignment & Padding:-** Structuring output with alignment and spacing.
4. **Decimal precision:-** Controls the number of decimal places for floating point numbers.
5. **Better Readability:-** Ideal for displaying tabular or structured data.

Explanation of Format Specifiers

%s → String

%d → Integer.

%f → Floating-point (default 6 decimal places)

%.2f → Floating-point with 2 decimal places

%.n → New line (preferred over \n for platform)

Example

```
System.out.printf("%d * %d = %d", 5, 7, 5*7).println()
```

↓

5*7=35

Introduction to Variables

int a = 10; → Values

↓ ↓

Data type Variables name

int b = 20; → This Statement is called a declaration. The declaration of

the variables declares what type it is and what is the name of the variables. So the declaration says the type of this variable is int and name is b and also declares a value [20].

example

`int i = 10;`

`System.out.printf("%.d * %.d = %.d", 5, i, 5 * i).println()`

↓ output
 $5 \times 10 = 50$

Exercises

1. Create three variables a, b & c. create a statement for printing sum of three variables. Modify value of a. Print value of a+b+c. Modify value of b. Print value of a+b+c.

Solution

`int a = 10; int b = 20; int c = 30`

~~`System.out.printf("%.d + %.d + %.d = %.d", a, b, c, a+b+c)`~~

`System.out.printf("%.d + %.d + %.d = %.d", a, b, c, a+b+c)`

↓ output

$10 + 20 + 30 = 60$

i) Modify value of A

`a = 20;`

`System.out.printf("%.d + %.d + %.d = %.d", a, b, c, a+b+c).println()`

↓

$20 + 20 + 30 = 70$

ii) Modify Value of B

$b = 10;$

System.out.printf("%d + %d + %d = %d", a, b, c, a+b+c).println()

↓

$$20 + 10 + 30 = 60$$

Few Puzzles related to Variables

1. `int number = 5.5;` → error. [incompatible types]

* Error because. The Data type is `int`. The Variable Value is float Value.

* Java is Strongly typed Language. by that means That you have to define a type for all Variables

* So, all Variable need to say. I am an integer, I am a string, I am a floating point Value, and you can only store that kind of Value is there

2. `int b = "Abcd";` → error.

* I cannot say B is a `int` and try to store a string Value [you cannot store string into a integer]

* Because Java is Strongly typed Language

3. `int a = 5;`

* I can use ~~name~~ A to create another variable

`int b = a;` → 5

* The value of A gets copied into B.

Variables in Depth

- Variables & Memory
- Naming a Variable
- Variable Types
- Assignment operators

→ How Variables Stored in memory

i) I am declaring a variable `int A` is equal to 5 and `B` is equal to 10

How do they get stored in memory

Memory Location	Value	Name
5004		
5008	5	A
5012	10	B
5016		
5020		
5024		
5028		

- * So, let's say A gets stored in the memory location 5008.
 - * And B gets stored in 5012.
 - * You have number of memory locations based on the size of your computer memory.
 - * The ~~etc~~
 - * Any Variable has a Name, it has a Value, and it also has a specific location in memory where it is stored.
 - * So the name is what we use in our program to refer to this memory location.
 - * So we are using a name A to refer to the location 5008.
 - * We are using a name B to refer to the memory location 5012.
 - * The way you can think about a Variable is just like a box, a box which has a Name, Value, Memory location.
 - * The other thing that Variable also has is a type, in java you need to declare the type of the Variable.
 - * You cannot change the type of a Variable during the execution of the program, and also you cannot store values of other type into this Variable.
- Example:

If A is an integer variable, you can only store integer numbers, you cannot store floating-point

→ How to name a Variable?

Naming a Variable Rules

- * Combination of letters, numbers, \$ and under-score (-)
- * Cannot start with a number
- * Cannot be a keyword
- * No limit on length of identifier
- * Camel case [best practice in java]

Need to

follow strictly

Example:-

1. int test - Test → Variable name

→ X error [It's not a legal Variable because hyphen usually stands for minus]

2. int 2test → Variable name

→ X error [Cannot start with a number]

3. int int → Variable name

→ X error [Cannot use a keyword]

4. int noOfOracles → ✓ Every word start with Capital letter except for the first one. First one start always lower case. [Camel Case]
This is a best practice in java

→ Understanding Primitive Variable Types

There are four different integer type

→ byte [8 bits] bits [-128 to 127] → Range of Variable

→ Short [16 bits] -32,768 to 32,767

→ int [32 bits] -2,147,483,648 to 2,147,483,647

→ Long [64 bits] → Range of long Variable

-9,223,372,036,854,775,808 to

9,223,372,036,854,775,807

There are two different floating type

→ float (f = 4.0f) [32 bits] 1.40239846e-45f to

3.40282347e+38f

→ Double (d = 67.0) [64 bits] 4.940656458412e-324

to 1.79769313486231570e+308

Other two type in java are

→ Character data type

→ Char (c = 'a') [16 bits] '\u0000' to '\uffff'

→ Boolean data type

→ boolean [true or false]

True and False always in lower case
like (true and false) as this
case sensitive

String Concatenation

Example

1. * Within a double quote is considered to be a string [String A = "wpsw"; → "wpsw"]

2. String a = "1" + 2; → "12"
Concatenated

[+]

* The Plus actually plays a double role. When you are putting plus around integers then it doing Addition

[int A = 1 + 2; → 3]

* When you put plus in a around String then it doing Concatenation

[String A = "123" + 45; → "12345"]

Example

1. 1 + 2 + 3 → 6

2. "1" + 2 + 3 → "123"

"The operations are always executed from left to right"

3. "1" + (2+3);
"1" + 5 → "15"

4. "ABC" + "DEF" → "ABCDEF"

5. System.out.println("ABC" + "DEF" + "FHI")
↓
ABCDEFHI

6. int i = 5
System.out.println("Value of I = " + i)
↓
Value of I = 5